

```
#!/usr/bin/env python3
```

```
"""
```

```
divergence_detector.py
```

```
-----
```

NovusÆxenti / NovÆcopia Autonomous Orchestration & Auditing Stack

Component: Divergence Detector & Policy Violation Watchdog

Responsibilities:

1. Evaluates incoming user prompt requirements against actual agent behavior and tool invocations.
2. Flags when an agent bypasses an available system tool, ignores hot-loaded skills, or attempts unpermitted moves/deletions.
3. Fires instant visual/toast red-flag alerts to the live dashboard ("Eyes in the Sky").
4. Logs structured divergence traces into 05_episodic_logs/divergence_incidents.jsonl.

```
"""
```

```
import os
```

```
import sys
```

```
import json
```

```
import re
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
LOG_DIR =
```

```
Path(os.path.expanduser("~/novae-xorpus/05_episodic_logs/divergence_incidents"))
```

```
class DivergenceIncident:
```

```
    def __init__(self, rule_id: str, severity: str, message: str, prompt_context: str, proposed_action: dict):
```

```
        self.incident_id = f"DIV-{{int(datetime.now(timezone.utc).timestamp()*1000)}}"
```

```
        self.timestamp = datetime.now(timezone.utc).isoformat()
```

```
        self.rule_id = rule_id
```

```
        self.severity = severity
```

```
        self.message = message
```

```
        self.prompt_context = prompt_context
```

```
        self.proposed_action = proposed_action
```

```
    def to_dict(self) -> dict:
```

```
        return {
```

```
            "incident_id": self.incident_id,
```

```
            "timestamp": self.timestamp,
```

```
            "rule_id": self.rule_id,
```

```
            "severity": self.severity,
```

```

        "message": self.message,
        "prompt_context": self.prompt_context,
        "proposed_action": self.proposed_action
    }

```

class DivergenceDetector:

```

    def __init__(self, strict_mode: bool = False):
        self.strict_mode = strict_mode
        self.registered_tools = {
            "manifest_compilation": ["compile_manifest.py", "generate_jsonl_markers.py"],
            "ast_analysis": ["graph_builder.py", "visual_topology_map.md"],
            "model_switching": ["npu_manager.py"],
            "hygiene_maintenance": ["system_housekeeper.sh", "markor_sweeper.py"],
            "corpus_extraction": ["doc_to_skill_and_tool.py", "convert_raw_to_markdown.py"]
        }
        self.protected_paths = [
            "__Nov/Exorpus_LIVING_MASTER_CANON",
            "OPERATOR_README.md",
            "OPERATOR_MAP.md",
            "00_DEFINITIVE_MASTER_SPECIFICATION_V3_COMPLETE.md"
        ]

```

```

    def check_file_protection(self, command_or_path: str) -> tuple[bool, str]:
        destructive_verbs = ["rm ", "rmdir", "mv ", "unlink", "truncate", "> ", ">> "]
        cmd_lower = command_or_path.lower()

```

```

        for verb in destructive_verbs:
            if verb in cmd_lower:
                for prot in self.protected_paths:
                    if prot.lower() in cmd_lower:
                        return False, f"Attempted destructive action '{verb.strip()}' against protected canonical path '{prot}'."
                return True, ""

```

```

    def check_tool_bypass(self, user_intent: str, proposed_tool: str, proposed_params: dict) -> tuple[bool, str]:
        intent_lower = user_intent.lower()
        raw_cmd = proposed_params.get("command", "")

```

```

        if any(w in intent_lower for w in ["compile manifest", "update manifest", "rebuild hash", "hash manifest"]):
            if "compile_manifest.py" not in raw_cmd and proposed_tool in ["bash", "sh"]:
                if "sha256" in raw_cmd or "md5" in raw_cmd or "find " in raw_cmd:

```

```
        return False, "Agent bypassed existing 'compile_manifest.py' tool with ad-hoc manual bash script."
```

```
    if any(w in intent_lower for w in ["ast review", "ast graph", "code review graph"]):
        if "graph_builder.py" not in raw_cmd and proposed_tool in ["bash", "sh"]:
            return False, "Agent bypassed 'code-review-graph/graph_builder.py' during requested AST structural analysis."
```

```
    return True, ""
```

```
def evaluate(self, user_intent: str, proposed_tool: str, proposed_params: dict) -> DivergenceIncident | None:
```

```
    raw_cmd = proposed_params.get("command", "") or json.dumps(proposed_params)
```

```
    safe, reason = self.check_file_protection(raw_cmd)
```

```
    if not safe:
```

```
        incident = DivergenceIncident(
            rule_id="INVARIANT_ZERO_TRUST_PROTECTION",
            severity="CRITICAL_HALT",
            message=reason,
            prompt_context=user_intent,
            proposed_action={"tool": proposed_tool, "params": proposed_params}
        )
        self._record_incident(incident)
        return incident
```

```
    safe, reason = self.check_tool_bypass(user_intent, proposed_tool, proposed_params)
```

```
    if not safe:
```

```
        incident = DivergenceIncident(
            rule_id="RULE_UNAUTHORIZED_TOOL_BYPASS",
            severity="WARNING" if not self.strict_mode else "CRITICAL_HALT",
            message=reason,
            prompt_context=user_intent,
            proposed_action={"tool": proposed_tool, "params": proposed_params}
        )
        self._record_incident(incident)
        return incident
```

```
    return None
```

```
def _record_incident(self, incident: DivergenceIncident):
```

```
    try:
```

```
        LOG_DIR.mkdir(parents=True, exist_ok=True)
```

```
log_file = LOG_DIR / "divergence_log.jsonl"
with open(log_file, "a", encoding="utf-8") as f:
    f.write(json.dumps(incident.to_dict(), ensure_ascii=False) + "\n")
    print(f"\n🚨 [DIVERGENCE DETECTED] [{incident.severity}] {incident.message}\n")
except Exception as e:
    sys.stderr.write(f"[Divergence Log Error] {e}\n")

if __name__ == "__main__":
    detector = DivergenceDetector()
    print("[*] Divergence Detector active.")
```